

Quantum Computing Assignment

Marya Alturki

August 2026

Code Availability The executable source code used for the computational results in this report is provided separately in the accompanying Jupyter Notebook (.ipynb) file. This report contains the answers, diagrams, results, explanations, and discussion for all three questions.

1 Question 1, Teleport the Letter A

1.1 Verification

Explain why quantum teleportation transfers a quantum state rather than physical matter and why classical communication is still required.

Quantum teleportation transfers the information contained in a quantum state from one qubit to another; it does not physically transport matter or the original particle. Alice and Bob first share an entangled pair of qubits. Alice performs a Bell-state measurement on the state to be teleported and her entangled qubit. This measurement destroys the original state at Alice's side. Bob's qubit can then be transformed into the original state using Alice's measurement results. Alice must send the two classical measurement results to Bob. These results tell Bob whether he needs to apply an X correction, a Z correction, or both to his qubit. Without this classical information, Bob cannot reconstruct the original state. Therefore, quantum teleportation requires both shared entanglement and classical communication and cannot be used for faster-than-light communication.

2 Question2 - BB84

2.1 Part A — The Security Proof

Read Shor and Preskill's paper, Simple Proof of Security of the BB84 Quantum Key Distribution Protocol, published in 2000. Then answer:

1. What security problem does the paper address?

The paper addresses the problem of proving that BB84 is secure against a general eavesdropper who can perform any physical operation allowed by quantum mechanics. The goal is to show that Eve can obtain only an exponentially small amount of information about the secret key.

2. How do the authors prove that BB84 is secure? Explain the roles of entanglement purification, the Lo-Chau security argument, and Calderbank-Shor-Steane (CSS) quantum error-correcting codes.

Shor and Preskill prove BB84 security by first considering an entanglement-based protocol and using entanglement purification to reduce noisy EPR pairs to nearly perfect ones. The Lo–Chau argument shows that when purification succeeds, Eve’s information about the key is exponentially small. They then use CSS quantum error-correcting codes, which separately correct bit- and phase-flip errors, to transform this entanglement-based protocol into the ordinary BB84 protocol. Therefore, the security of the purified entanglement protocol implies the security of BB84.

2.2 Part B — A Modern Analysis

Read the 2023 paper, Comprehensive Analysis of BB84, A Quantum Key Distribution Protocol. In shore answer (few sentences), state the paper’s main finding about the security, limitations, improvements, and practical use of BB84.

The paper concludes that BB84 is a highly secure and extensively studied QKD protocol, but practical implementations still face limitations such as imperfect detectors, non-ideal single-photon sources, and susceptibility to some attacks. It discusses improvements such as decoy states and advanced error-correction techniques to increase security and practicality. The paper also highlights practical applications of BB84/QKD in communication networks, IoT, and real-time systems.

3 Question3 - Quantum Error Correction

The objective is to implement the three-qubit bit-flip quantum error-correcting code to protect one logical qubit, diagnose a single Pauli- X error, apply the appropriate correction, decode the logical qubit, and compare protected and unprotected transmission.

The implemented sequence is

prepare $\rightarrow$ encode $\rightarrow$ inject error $\rightarrow$ extract syndrome $\rightarrow$ correct $\rightarrow$ decode $\rightarrow$ measure.

The required protected inputs are $|0\rangle$, $|1\rangle$, and the superposition state $|+\rangle$. Each is tested for no error and for a single X error on physical qubits q_0 , q_1 , and q_2 .

4 Physical and Computational Problem

A general qubit is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

The bit-flip error is represented by

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$

with

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle.$$

The three-qubit repetition code encodes one logical qubit into three physical qubits according to

$$|0_L\rangle = |000\rangle, \quad |1_L\rangle = |111\rangle.$$

Therefore,

$$\boxed{|\psi_L\rangle = \alpha|000\rangle + \beta|111\rangle.}$$

The three data qubits are q_0, q_1, q_2 . Two ancilla qubits a_0, a_1 record parity information used for syndrome detection.

5 Mathematical Formulation

5.1 Encoding

The initial state is

$$(\alpha|0\rangle + \beta|1\rangle) \otimes |0\rangle \otimes |0\rangle = \alpha|000\rangle + \beta|100\rangle.$$

Applying $\text{CNOT}_{0 \rightarrow 1}$ gives

$$\alpha|000\rangle + \beta|110\rangle,$$

and applying $\text{CNOT}_{0 \rightarrow 2}$ gives

$$\boxed{\alpha|000\rangle + \beta|111\rangle.}$$

Thus,

$$|0\rangle \rightarrow |000\rangle, \quad |1\rangle \rightarrow |111\rangle,$$

and

$$|+\rangle \rightarrow \frac{|000\rangle + |111\rangle}{\sqrt{2}}.$$

5.2 Single- X errors

Define

$$X_0 = X \otimes I \otimes I, \quad X_1 = I \otimes X \otimes I, \quad X_2 = I \otimes I \otimes X.$$

The resulting states are

$$X_0|\psi_L\rangle = \alpha|100\rangle + \beta|011\rangle, \tag{1}$$

$$X_1|\psi_L\rangle = \alpha|010\rangle + \beta|101\rangle, \tag{2}$$

$$X_2|\psi_L\rangle = \alpha|001\rangle + \beta|110\rangle. \tag{3}$$

5.3 Syndrome operators

The Pauli- Z operator satisfies

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle.$$

For two qubits,

$$(Z \otimes Z)|00\rangle = +|00\rangle, \quad (Z \otimes Z)|11\rangle = +|11\rangle,$$

whereas

$$(Z \otimes Z)|01\rangle = -|01\rangle, \quad (Z \otimes Z)|10\rangle = -|10\rangle.$$

Hence the two parity checks are

$$S_1 = Z_0 Z_1, \quad S_2 = Z_1 Z_2.$$

In the implemented Qiskit circuit, the ancilla bits store

$$s_0 = q_0 \oplus q_1, \quad s_1 = q_1 \oplus q_2.$$

The relation between the measured bit and the ZZ eigenvalue is

$$s = 0 \leftrightarrow +1, \quad s = 1 \leftrightarrow -1.$$

The resulting syndrome map used in the implementation is

$$I \rightarrow 00, \quad X_0 \rightarrow 01, \quad X_1 \rightarrow 11, \quad X_2 \rightarrow 10.$$

6 Circuit Implementation

6.1 Encoding circuit

The encoding circuit uses two CNOT gates. The first CNOT has q_0 as the control and q_1 as the target, and the second has q_0 as the control and q_2 as the target. The circuit therefore implements

$$\alpha|0\rangle + \beta|1\rangle \longrightarrow \alpha|000\rangle + \beta|111\rangle.$$

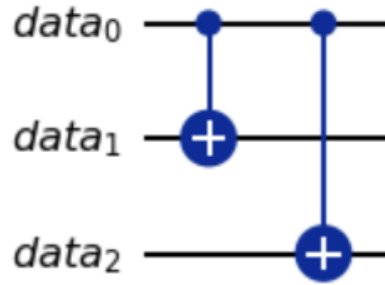


Figure 1: Encoding circuit used in the notebook. The circuit applies $\text{CNOT}_{0 \rightarrow 1}$ followed by $\text{CNOT}_{0 \rightarrow 2}$.

6.2 Syndrome-detection circuit

The two ancilla qubits accumulate the parity information $q_0 \oplus q_1$ and $q_1 \oplus q_2$. They are then measured and the results are stored in the two-bit classical syndrome register.

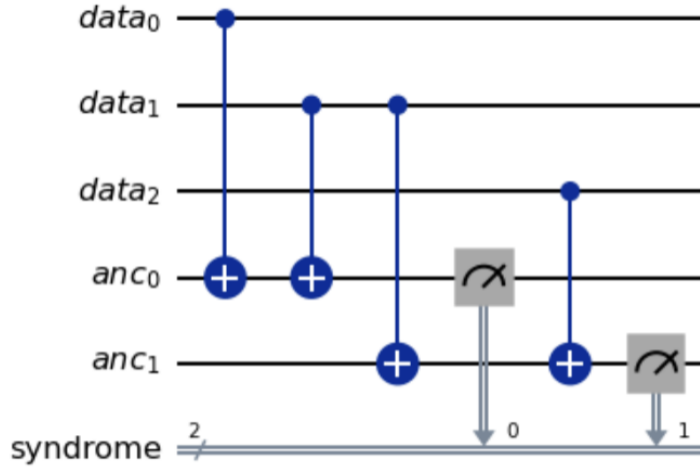


Figure 2: Syndrome-extraction circuit generated in the notebook. The ancillas record the two parity checks before measurement into the two-bit classical syndrome register.

6.3 Conditional correction circuit

The measured syndrome determines which physical qubit is corrected. In the Qiskit implementation, the mapping is

$$00 \rightarrow I, \quad 01 \rightarrow X_0, \quad 11 \rightarrow X_1, \quad 10 \rightarrow X_2.$$

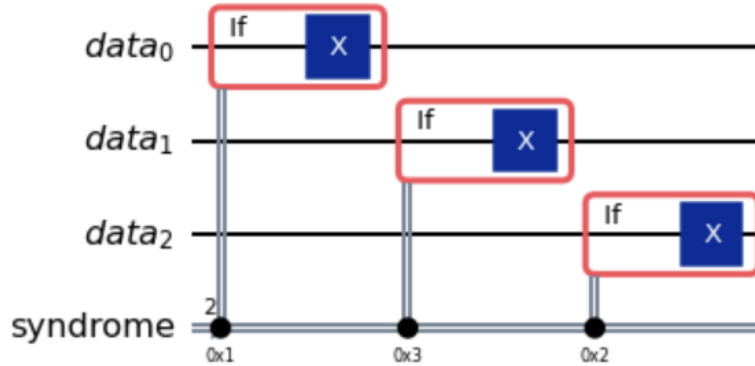


Figure 3: Conditional correction stage generated in the notebook. The appropriate X gate is applied according to the measured syndrome.

6.4 Decoding circuit

The encoding is inverted using the CNOTs in reverse order.

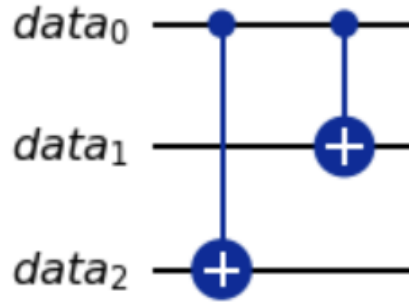


Figure 4: Decoding circuit used in the notebook. The logical information is recovered in q_0 .

6.5 Complete QEC circuit

The notebook also generated a complete circuit for the representative test $|+\rangle$ with a deliberately injected X_1 error.

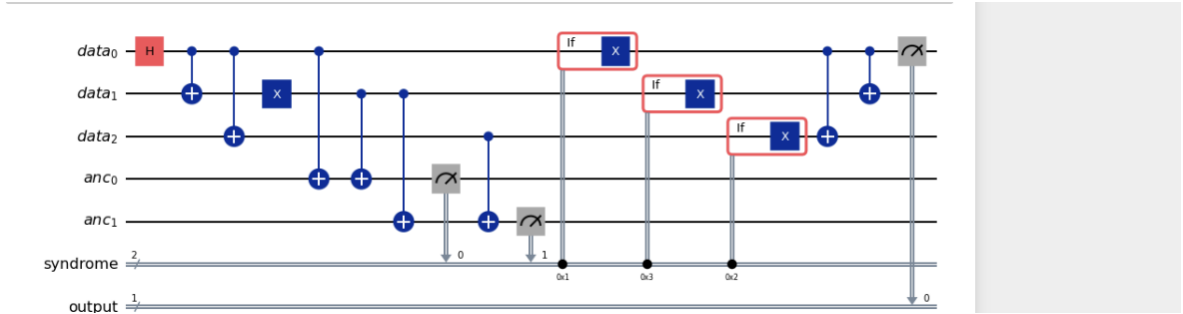


Figure 5: Complete QEC circuit for the representative $|+\rangle$ input with an injected X_1 error. The displayed circuit is the same Qiskit circuit constructed by `build_qec_circuit("+",1)`.

7 Qiskit Source Code

The following is the final reusable circuit-construction code used in the notebook, including the same input states, error locations, syndrome mapping, and conditional corrections.

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from IPython.display import display
```

```
def build_qec_circuit(input_state, error_location=None):
    """
    Build a three-qubit bit-flip QEC circuit.
```

Parameters

```
input_state : str
    "0", "1", or "+".
error_location : int or None
    None → no error
    0     → X error on data qubit 0
    1     → X error on data qubit 1
    2     → X error on data qubit 2
```

Returns

```
QuantumCircuit
    Complete QEC circuit.
"""

data = QuantumRegister(3, "data")
anc = QuantumRegister(2, "anc")
syndrome = ClassicalRegister(2, "syndrome")
output = ClassicalRegister(1, "output")

qc = QuantumCircuit(data, anc, syndrome, output)

# 1. Prepare input state
if input_state == "0":
    pass
elif input_state == "1":
    qc.x(data[0])
elif input_state == "+":
    qc.h(data[0])
else:
    raise ValueError(
        "input_state must be '0', '1', or '+'."
    )

# 2. Encode
qc.cx(data[0], data[1])
qc.cx(data[0], data[2])

# 3. Inject selected X error
if error_location is None:
    pass
elif error_location in (0, 1, 2):
    qc.x(data[error_location])
else:
    raise ValueError(
        "error_location must be None, 0, 1, or 2."
    )
```

```

# 4. Syndrome extraction
# anc[0] = q0 XOR q1
# anc[1] = q1 XOR q2
qc.cx(data[0], anc[0])
qc.cx(data[1], anc[0])
qc.cx(data[1], anc[1])
qc.cx(data[2], anc[1])

# 5. Measure syndrome
qc.measure(anc[0], syndrome[0])
qc.measure(anc[1], syndrome[1])

# 6. Conditional correction
# Qiskit register values:
# 01 -> X0, 11 -> X1, 10 -> X2
with qc.if_test((syndrome, 0b01)):
    qc.x(data[0])
with qc.if_test((syndrome, 0b11)):
    qc.x(data[1])
with qc.if_test((syndrome, 0b10)):
    qc.x(data[2])

# 7. Decode
qc.cx(data[0], data[2])
qc.cx(data[0], data[1])

# 8. Measure recovered logical qubit
qc.measure(data[0], output[0])

return qc

```

```

from qiskit_aer import AerSimulator

```

```

def build_unprotected_circuit(input_state, error_location):
    """
    Build an unprotected single-qubit circuit.

    The unprotected experiment has only one physical qubit, so only
    error_location=0 is meaningful when an X error is requested.
    """

    qubit = QuantumRegister(1, "q")
    output = ClassicalRegister(1, "output")

    qc = QuantumCircuit(qubit, output)

```

```

# Prepare input state
if input_state == "0":
    pass
elif input_state == "1":
    qc.x(qubit[0])
elif input_state == "+":
    qc.h(qubit[0])
else:
    raise ValueError(
        "input_state must be '0', '1', or '+'."
    )

# Apply X error to the only physical qubit
if error_location is not None:
    if error_location != 0:
        raise ValueError(
            "An unprotected circuit has only one physical qubit: q0."
        )
    qc.x(qubit[0])

# Measure
qc.measure(qubit[0], output[0])

return qc

def run_circuit(qc, shots=1000):
    """Run a Qiskit circuit on an ideal shot-based simulator."""
    simulator = AerSimulator()
    result = simulator.run(qc, shots=shots).result()
    return result.get_counts()

```

8 Verification and Results

All protected experiments shown below used 1000 shots on an ideal **AerSimulator**. No hardware noise model was added; the only errors were the deliberately injected single- X errors.

8.1 Protected $|0\rangle$ experiments

The notebook produced

Input	Error	Syndrome	Observed result
$ 0\rangle$	None	00	1000/1000 $\rightarrow$ 0
$ 0\rangle$	X_0	01	1000/1000 $\rightarrow$ 0
$ 0\rangle$	X_1	11	1000/1000 $\rightarrow$ 0
$ 0\rangle$	X_2	10	1000/1000 $\rightarrow$ 0

8.2 Protected $|1\rangle$ experiments

The notebook produced

Input	Error	Syndrome	Observed result
$ 1\rangle$	None	00	1000/1000 $\rightarrow$ 1
$ 1\rangle$	X_0	01	1000/1000 $\rightarrow$ 1
$ 1\rangle$	X_1	11	1000/1000 $\rightarrow$ 1
$ 1\rangle$	X_2	10	1000/1000 $\rightarrow$ 1

8.3 Protected $|+\rangle$ experiments

For $|+\rangle$, the expected final measurement probabilities are

$$P(0) = P(1) = \frac{1}{2}.$$

The final systematic 1000-shot run shown in the notebook gave

Input	Error	Syndrome	0 counts	1 counts
$ +\rangle$	None	00	499	501
$ +\rangle$	X_0	01	503	497
$ +\rangle$	X_1	11	485	515
$ +\rangle$	X_2	10	539	461

Equivalently,

Error	$P(0)$	$P(1)$
None	0.499	0.501
X_0	0.503	0.497
X_1	0.485	0.515
X_2	0.539	0.461

These deviations from 0.5 are finite-shot sampling fluctuations. Each syndrome remained deterministic for the corresponding error case.

8.4 Complete protected results table

Input	Error	Syndrome	Expected	Observed
$ 0\rangle$	None	00	0	1000 $\rightarrow$ 0
$ 0\rangle$	X_0	01	0	1000 $\rightarrow$ 0
$ 0\rangle$	X_1	11	0	1000 $\rightarrow$ 0
$ 0\rangle$	X_2	10	0	1000 $\rightarrow$ 0
$ 1\rangle$	None	00	1	1000 $\rightarrow$ 1
$ 1\rangle$	X_0	01	1	1000 $\rightarrow$ 1
$ 1\rangle$	X_1	11	1	1000 $\rightarrow$ 1
$ 1\rangle$	X_2	10	1	1000 $\rightarrow$ 1
$ +\rangle$	None	00	$ +\rangle$	0:499, 1:501
$ +\rangle$	X_0	01	$ +\rangle$	0:503, 1:497
$ +\rangle$	X_1	11	$ +\rangle$	0:485, 1:515
$ +\rangle$	X_2	10	$ +\rangle$	0:539, 1:461

Note on the $|+\rangle$ counts. Because $|+\rangle$ is measured probabilistically, independent 1000-shot runs need not give the same frequencies. The values reported above are the final systematic $|+\rangle$ run in the notebook (the loop over None, X_0 , X_1 , X_2), rather than an earlier standalone test. This avoids mixing independent shot samples.

9 Protected Versus Unprotected Transmission

The unprotected control circuit contains one physical qubit. Therefore, there is only one meaningful physical error location, q_0 .

The notebook measured

$$|0\rangle \xrightarrow{X} |1\rangle, \quad \text{result } \{ '1' : 1000 \},$$

and

$$|1\rangle \xrightarrow{X} |0\rangle, \quad \text{result } \{ '0' : 1000 \}.$$

The protected experiments instead recovered $|0\rangle$ and $|1\rangle$ after X_0 , X_1 , or X_2 .

Thus, for these deliberately injected bit-flip errors in the ideal simulation,

$$P_{\text{correct}}^{\text{unprotected}} = 0, \quad P_{\text{correct}}^{\text{protected}} = 1,$$

for the two computational-basis examples.

For $|+\rangle$, the unprotected comparison is not diagnostically useful at the level of computational-basis measurement because

$$X|+\rangle = |+\rangle.$$

10 Investigation

10.1 Correctable error patterns

The code corrects the no-error case and any single-qubit X error:

$$\boxed{I, X_0, X_1, X_2.}$$

Each has a unique syndrome, so the recovery operation is unambiguous within this error model.

10.2 An error pattern the code cannot correct

A simple example is the two-error pattern

$$X_0X_1.$$

For the encoded state,

$$X_0X_1(\alpha|000\rangle + \beta|111\rangle) = \alpha|110\rangle + \beta|001\rangle.$$

Its syndrome is 10, which the single-error decoder interprets as X_2 . Thus a two-error event can be misidentified as a single error, so the present code cannot generally correct multiple simultaneous bit flips.

10.3 Bit-flip versus phase-flip

A bit flip exchanges computational-basis states:

$$X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle.$$

A phase flip leaves the basis states in place but changes their relative phase:

$$Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle.$$

For example,

$$Z|+\rangle = |-\rangle.$$

The three-qubit repetition code is specifically constructed to detect X -type errors through its parity checks and does not by itself correct arbitrary Z errors.

10.4 Protecting against both error types

A code protecting against both X - and Z -type errors requires a more general quantum error-correcting structure. Examples include CSS codes, the Shor nine-qubit code, and surface codes. The three-qubit repetition code is therefore not a universal QEC code.

11 No-Cloning Theorem

Classical repetition can copy a classical bit,

$$0 \rightarrow 000, \quad 1 \rightarrow 111,$$

but there is no universal quantum operation that maps an arbitrary unknown state to three independent copies.

The QEC encoding instead uses

$$\boxed{\alpha|0\rangle + \beta|1\rangle \rightarrow \alpha|000\rangle + \beta|111\rangle.}$$

For example,

$$|+\rangle^{\otimes 3} = \frac{1}{2\sqrt{2}} (|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle),$$

whereas the three-qubit QEC encoding gives

$$|+_L\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}.$$

Therefore,

$$\boxed{|+\rangle^{\otimes 3} \neq \frac{|000\rangle + |111\rangle}{\sqrt{2}}}.$$

12 Conclusion

The three-qubit bit-flip code was implemented in Qiskit and tested for the three required input states, the no-error case, and a single X error on each physical data qubit. The measured syndromes matched the predicted mapping

$$I \rightarrow 00, \quad X_0 \rightarrow 01, \quad X_1 \rightarrow 11, \quad X_2 \rightarrow 10.$$

For $|0\rangle$ and $|1\rangle$, all required protected cases returned the original logical state in all 1000 shots. For $|+\rangle$, the final probabilities remained close to $1/2$ for each outcome, as expected for the recovered superposition state.

The unprotected control experiments showed that a single X error directly changes the computational-basis logical state, while the protected circuit recovers the original state under the assigned error model. The investigation further established that the repetition code is limited to single bit-flip errors and does not itself provide general protection against arbitrary quantum errors.